

Practica 12: Cadenas de Markov

En esta práctica aprenderemos a simular Cadenas de Markov y con ello analizaremos el comportamiento de varios ejemplos.

Operaciones con Matrices en Python

El modulo de algebra lineal de numpy nos ofrece una gran cantidad de rutinas para trabajar con matrices y vectores. La primera función que usaremos será

`np.linalg.matrix_power()`, la cual nos ayudará a elevar una matriz A a la potencia n . Para llevar a cabo esta operación simplemente hay que llamar a la función de la siguiente manera:

```
np.linalg.matrix_power(A,n)
```

Otra función que nos será de utilidad es `np.dot()`, la cual podemos usar para calcular productos entre matrices. En particular nosotros la usaremos para calcular el producto de un vector, v , con una matriz, A . Para llevar a cabo esta operación simplemente hay que llamar a la función de la siguiente manera:

```
np.dot(A, v)
```

Recordemos que las dimensiones importan a la hora de llevar a cabo el producto. Veamos que la función `np.dot()` respeta esto:

```
# Un vector de 1 x 3
v = np.array([1,0,1]).reshape(1,3)
print(v)
```

```
# Una matriz de 3 x 3
A = np.array([2,1,8,
              2,5,0.4,
              4,0,7]).reshape(3,3)
```

```
print(A)
print('=====')
```

```
# El producto vA está bien definido
print(np.dot(v,A))
```

```
[[1 0 1]]
[[2.  1.  8. ]
 [2.  5.  0.4]
 [4.  0.  7. ]]
=====
[[ 6.  1. 15.]]
```

```
# Un vector de 3 x 1
v = np.array([1,0,1]).reshape(3,1)
print(v)
```

```
# Una matriz de 3 x 3
A = np.array([2,1,8,
              2,5,0.4,
              4,0,7]).reshape(3,3)

print(A)
print('=====')
# El producto vA regresa un error
print(np.dot(v,A))
```

```
[[1]
 [0]
 [1]]
[[2.  1.  8. ]
 [2.  5.  0.4]
 [4.  0.  7. ]]
=====
```

```
-----
-----
ValueError                                Traceback (most recent call
last)
<ipython-input-17-499ac813c225> in <module>
      10 print('=====')
      11 # El producto vA regresa un error
--> 12 print(np.dot(v,A))
```

```
<__array_function__ internals> in dot(*args, **kwargs)
```

ValueError: shapes (3,1) and (3,3) not aligned: 1 (dim 1) != 3 (dim 0)

Ejercicio 1:

Escribe las matrices faltantes como un arreglo en Python y eleva todas las matrices a la potencia n con $n=10,100,1000$

$$P_1 = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix}, P_2 = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & \frac{99}{200} & \frac{1}{200} \\ 0 & 0 & 1 \end{pmatrix}, P_3 = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ \frac{1}{4} & 0 & \frac{3}{4} & 0 & 0 \\ 0 & \frac{2}{4} & 0 & \frac{2}{4} & 0 \\ 0 & 0 & \frac{3}{4} & 0 & \frac{1}{4} \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}, P_4 = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0.6 & 0 & 0.4 & 0 & 0 & 0 \\ 0 & 0.6 & 0 & 0.4 & 0 & 0 \\ 0 & 0 & 0.6 & 0 & 0.4 & 0 \\ 0 & 0 & 0 & 0.6 & 0 & 0.4 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

```
# P1 =

# P2 =

P3 = np.array([0,1,0,0,0,0,
               1/5,0,4/5,0,0,0,
               0,2/5,0,3/5,0,0,
               0,0,3/5,0,2/5,0,
               0,0,0,4/5,0,1/5,
               0,0,0,0,1,0]).reshape(6,6)

P4 = np.array([1,0,0,0,0,0,
               0.5,0,0.5,0,0,0,
               0,0.5,0,0.5,0,0,
               0,0,0.5,0,0.5,0,
               0,0,0,0.5,0,0.5,
               0,0,0,0,0,1]).reshape(6,6)
```

Ejercicio 2:

Haz una función que **simule en estado dado una distribución** de una cadena de Markov considerando una distribución inicial.

$$v = (p_1, p_2, \dots, p_n)$$

Si numeramos los estados de la cadena comenzando con 0 la idea es exactamente la misma que cuando programaron el dado cargado en la **práctica 10**.

Su función debe recibir:

- un vector de probabilidades cuya suma sea 1

Debe regresar de manera aleatoria (con la probabilidad de elección correspondiente) una de las etiquetas de los estados.

Idea para simular una cadena de Markov

Necesitamos generar una sucesión $[x_0, x_1, x_2, \dots, x_N]$ dado un espacio de estados E .

Supongamos un espacio de estados $E = \{0, 1, 2\}$, $\pi = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$ y $P = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix}$.

1. Simulamos x_0 . Para esto necesitamos el vector de distribución inicial $\pi = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$.

Escogemos un estado aleatorio con la función anterior y π . Listo! tenemos x_0 supongamos que $x_0=2$.

1. Simulamos x_1 . Una cadena de Markov depende del estado en que estuve antes, entonces tenemos que utilizar la matriz de transición y fijarnos en el renglón del estado en que estábamos.

$$P[2,:]=\left(0, \frac{1}{2}, \frac{1}{2}\right)$$

Escogemos un estado aleatorio con la función anterior y $P[2,:]$.

Listo! Repetimos... hasta n .

Ejercicio 3:

En este ejercicio programarán dos funciones que simule una cadena de Markov y los estados que va tomando en cada iteración. La diferencia entre estas funciones será el criterio de paro.

- La primera función deberá hacer:
 - a. N iteraciones de la cadena de markov y la deberá recibir como parámetro.
 - b. Regresar la lista de los estados que recorrió.
- Mientras que la segunda función:
 1. deberá correr hasta que la cadena tome un estado e dado. Deberá recibir la etiqueta del estado al que se quiere llegar, y un número máximo de iteraciones, $MaxIter$, en caso de que nunca se alcance el estado deseado.

Ambas funciones deben recibir **dos parámetros correspondientes a la cadena**.

- El primer parámetro es deberá ser la matriz de transición de la cadena, P .
- El segundo parámetro deberá ser la distribución inicial de la cadena, Π_0 .

Ejercicio 4:

Prueba la función del ejercicio anterior con **las matrices del ejercicio 1**.

Prueba cada caso con **distribuciones iniciales aleatorias** usando las siguientes líneas de código:

```
#Generamos n uniformes(0, 1), donde n corresponde al numero de estados.
Pi_0 = np.random.random(n)
#Dividimos al vector entre su suma y asi el vector es una distribución de probabilidad.
Pi_0 /= Pi_0.sum()
```

Usa la primera función del ejercicio anterior con 100 iteraciones. Finalmente, grafica el vector de los estados recorridos por la cadena contra el tiempo.

Cadena 1

Es el clásico ejemplo de los amigos que comparten una botella de agua de horchata y la van rotando tirando volados.

El del centro siempre la pasa a los amigos de los costados y los amigos de los costados siempre la regresan al centro o se la quedan por un turno. Esta cadena es irreducible.

Estados $\{A, B, C\}$

$$P_1 = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix}$$

```
import matplotlib.pyplot as plt
```

#Tu código para simular

Cadena 2

La segunda cadena de Markov es una modificación del ejemplo anterior.

En este caso podemos pensar que uno de los amigos es muy gandalla y no le quieren compartir porque saben que no la va a regresar y se la va a quedar para el solo. Sin embargo, uno de los amigos es un poco distraído y existe una pequeña posibilidad de que se distraiga y se la pase al gandalla. Esta cadena es reducible.

Estados $\{A, B, G\}$

$$P_2 = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & \frac{99}{200} & \frac{1}{200} \\ 0 & 0 & 1 \end{pmatrix}$$

#Tu código para simular

Tercer Cadena

La tercer cadena de Markov es la urna de Ehrenfest con 5 bolas.

Se tienen dos urnas, con 5 bolas repartidas dentro de ellas (numeradas), y en cada etapa se escoge una bola al azar y se cambia de urna. La cadena X_n representa el número de bolas de una de las urnas tras n etapas.

Estados $\{0, 1, 2, 3, 4, 5\}$

$$P_3 = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ \frac{1}{5} & 0 & \frac{4}{5} & 0 & 0 & 0 \\ 0 & \frac{2}{5} & 0 & \frac{3}{5} & 0 & 0 \\ 0 & 0 & \frac{3}{5} & 0 & \frac{2}{5} & 0 \\ 0 & 0 & 0 & \frac{4}{5} & 0 & \frac{1}{5} \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

#Tu código para simular

Cuarta Cadena

La última cadena de Markov es un ejemplo de la ruina del Jugador. En este caso se puede decir que están tirando volados para ver quien se lleva 5 monedas de 1 peso pero la apuesta es todo o nada. Entonces puede suceder que al final pierda y se quede con 0 pesos o gane y se quede con 5 pesos (o algún estado en medio).

Estados $\{0, 1, 2, 3, 4, 5\}$

$$P_4 = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0.5 & 0 & 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0.5 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0.5 & 0 & 0.5 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

#Tu código para simular

Ejercicio 5:

Podemos aproximar la probabilidad de que la cadena se encuentre en cada estado después de muchas iteraciones usando la distribución inicial Π_0 y la matriz de transición P .

Para ello basta hacer el producto $\Pi_0 P^n$ para valores de n grandes. Haz ese producto para cada matriz de transición utilizando $n=1000$ y diferentes tres distribuciones iniciales generadas de manera aleatoria.

¿La elección de distribución inicial afecta en algo?